

Functions: Important Questions and Answers

1. Define Function. Explain Different Types of Functions.

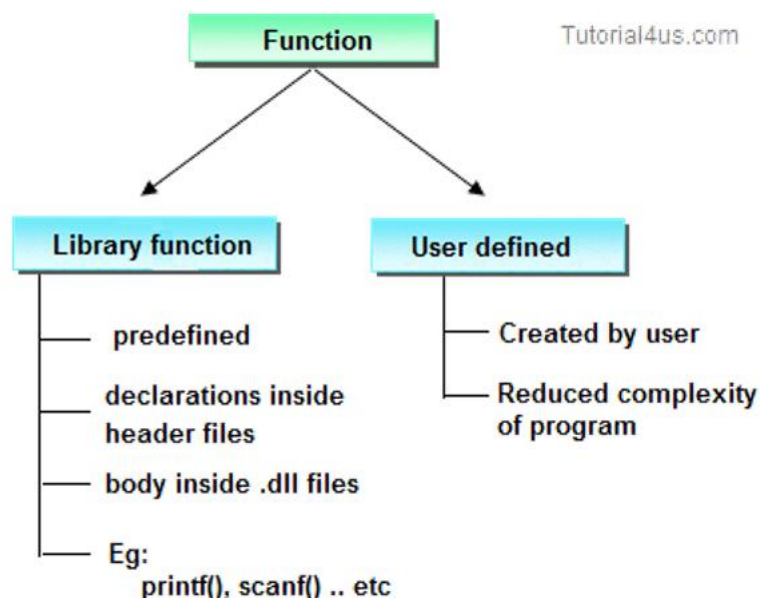
Function - Definition: A function is a block of code/group of statements/self contained block of statements/ basic building blocks in a program that performs a particular task. It is also known as procedure or subroutine or module, in other programming languages. To perform any task, we can create function. A function can be called many times. It provides modularity and code reusability.

Advantage of functions

- 1) Code Reusability By creating functions in C, you can call it many times. So we don't need to write the same code again and again.
- 2) Code optimization It makes the code optimized we don't need to write much code.
- 3) Easily to debug the program.

Types of Functions There are two types of functions in C programming:

1. **Library Functions:** are the functions which are declared in the C header files such as scanf(), printf(), gets(), puts(), ceil(), floor() etc. You just need to include appropriate header files to use these functions. These are already declared and defined in C libraries. To use system defined functions the respective header file must be included.
2. **User-defined functions:** are the functions which are created by the C programmer, so that he/she can use it many times. It reduces complexity of a big program and optimizes the code. Depending upon the complexity and requirement of the program, you can create as many user-defined functions as you want.



When a function is called, **parameters** allow data to be passed between the **calling function** and the **called function**.

1. Actual Parameters (Arguments)

Definition:

Actual parameters (or arguments) are the **values or variables passed to a function** when it is called.

Key Points:

- Appears in the **function call**.
- Can be **constants, variables, expressions, or function return values**.

Example:

```
#include <stdio.h>
```

```
void display(int n) {  
    printf("Number = %d\n", n);  
}
```

```
int main() {  
    int num = 10;  
    display(num); // num is the actual parameter  
    display(20); // 20 is also an actual parameter  
    return 0;  
}
```

2. Formal Parameters

Definition:

Formal parameters are **variables declared in the function definition** that **receive the values** of actual parameters.

Key Points:

- Appears in the **function definition**.
- Acts as **placeholders** to hold the value passed from the calling function.

Example:

```
void display(int n) // n is a formal parameter  
{  
    printf("Number = %d\n", n);  
}
```

Differences Between Actual and Formal Parameters

Actual Parameters	Formal Parameters
Values or variables passed to the function	Variables declared in the function to receive values
At the function call	In the function definition
Provides input to the function	Receives input from the calling function

Actual Parameters	Formal Parameters
display(10); display(num);	void display(int n)
Must match with formal parameters	Must match with actual parameters

4. Summary

- **Actual parameters:** Real values supplied to the function when called.
 - **Formal parameters:** Variables in the function definition that accept those values.
 - Both are used together to **pass data between functions**.
-

2. Discuss the types of functions based on input parameters and return values.

Functions can be classified in to four types based on the input parameters and return values.

- Functions with no Parameters and no Return Values.
- Functions with no Parameters and Return Values.
- Functions with Parameters and no Return Values.
- Functions with Parameters and Return Values.

Functions with no Parameters and no Return Values

- In this category, there is no data transfer between the calling function and called function.
- But there is flow of control from calling function to the called function.
- When no parameters are there , the function cannot receive any value from the calling function.
- When the function does not return a value, the calling function cannot receive any value from the called function.

```
#include <stdio.h>
```

```
// Function declaration
```

```
void displayMessage(); // no parameters, no return value
```

```
int main() {
```

```
    // Function call
```

```
    displayMessage();
```

```

    return 0;
}

// Function definition

void displayMessage() {

    printf("This is a function with no parameters and no return value!\n");

}

```

Functions with no Parameters and Return Values

- In this category, there is no data transfer between the calling function and called function.
- But there is data transfer from called function to the calling function.
- When no parameters are there , the function cannot receive any values from the calling function.
- When the function returns a value, the calling function receives one value from the called function.

Example:

```

#include <stdio.h>
// Function declaration
int add(int, int); // parameters and return value
int main() {
    int num1, num2, result;
    printf("Enter two numbers: ");
    scanf("%d %d", &num1, &num2);
    // Function call with parameters
    result = add(num1, num2);
    printf("Sum = %d\n", result);
    return 0;
}
// Function definition
int add(int a, int b)
{
    return a + b; // returns the sum
}

```

Functions with Parameters and no Return Values.

- In this category, there is data transfer from the calling function to the called function using parameters.
- But there is no data transfer from called function to the calling function.
- When parameters are there , the function can receive any values from the calling function.
- When the function does not return a value, the calling function cannot receive any value from the called function.

Example:

```

#include <stdio.h>
// Function declaration

```

```

void displaySum(int, int); // parameters, no return value
int main() {
    int num1, num2;
    printf("Enter two numbers: ");
    scanf("%d %d", &num1, &num2);
    // Function call with parameters
    displaySum(num1, num2);
    return 0;
}
// Function definition
void displaySum(int a, int b)
{
    int sum = a + b;
    printf("The sum of %d and %d is: %d\n", a, b, sum);
}

```

Functions with Parameters and Return Values.

- In this category, there is data transfer from the calling function to the called function using parameters.
- But there is no data transfer from called function to the calling function.
- When parameters are there, the function can receive any values from the calling function.
- When the function returns a value, the calling function receives a value from the called function.

Example:

```

#include <stdio.h>
// Function declaration
int multiply(int, int); // parameters and return value
int main() {
    int a, b, result;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    // Function call with parameters
    result = multiply(a, b);
    printf("Product = %d\n", result);
    return 0;
}
// Function definition
int multiply(int x, int y) {
    return x * y; // returns the product
}

```

Thus, the above examples explain the classification of functions based on the parameters and return values

3. Differentiate call by value and call by reference

Call by Value:

In **Call by Value**, a **copy of the actual parameter** is passed to the function. Any change made to the parameter **inside the function does NOT affect** the original variable.

Characteristics

- Actual value is copied to the function's formal parameter.
- Changes affect only the copy.
- Original data remains safe.
- Default mechanism in C.

Example:

```
void change(int x) {  
    x = 20; // modifies only the local copy  
}
```

Example:

```
int main()  
{  
    int a = 10;  
    change(a);  
    printf("%d", a); // Output: 10 (no change)  
}
```

Call By Reference (CBR)

In **Call by Reference**, the **address of the variable** is passed to the function. Any change made inside the function **directly affects the original variable**.

Characteristics

- Address (reference) is passed.
- Changes reflect on the original value.
- Requires pointers in C.
- Useful for modifying values or returning multiple results.

Example:

```
void change(int *x)  
{  
    *x = 20; // modifies the actual variable  
}  
int main()  
{  
    int a = 10;  
    change(&a);  
    printf("%d", a); // Output: 20 (value changed)  
}
```

Call By Value	Call By Reference
Passes a copy of the value	Passes the address of the variable
Do not affect original variable	Affect the original variable
Requires more memory (stores copies)	Requires less memory (no copying)
Safer (original data cannot be altered unintentionally)	Less safe (original data can be modified)
Default method	Implemented using pointers
Slightly slower (due to copying)	Faster (direct access via address)
When you don't want to modify original data	When you want to modify original data or return multiple values
func(a);	func(&a);

4. Explain the elements of a User Defined function.

Elements of a User-Defined Function

A **user-defined function** in C typically consists of the following major elements:

1. Function Name

- Identifies the function uniquely within the program.
- Follows the same naming rules as variables (letters, digits, underscore; cannot start with a digit).
- Should be meaningful and reflect the function's purpose.

Example:

```
int add(int x, int y);
```

Here, add is the function name.

2. Return Type

- Specifies the **type of value** the function returns to the calling function.
- Common return types: int, float, char, double, void.

Example:

```
int sum(); // returns integer
```

```
void display(); // returns nothing
```

3. Parameter List (Arguments)

- Input values passed to the function.
- Enclosed in parentheses ().
- Can be zero or more parameters.
- Each parameter must have a data type.

Example:

```
int add(int a, int b); // two parameters
void show();           // no parameters
```

4. Function Declaration (Prototype)

- Tells the compiler about the function name, return type, and parameters before its actual use.
- Helps prevent errors by enabling type checking.

Example:

```
int add(int, int);
```

5. Function Definition

This component contains the **actual implementation** of the function.

A function definition includes:

1. Return type
2. Function name
3. Parameter list
4. Function body

Example:

```
int add(int a, int b) {
    int c = a + b;
    return c;
}
```

6. Function Body

The body is the set of statements enclosed within { }.

It includes:

- Local variable declarations
- Logic/operations to perform the task
- return statement (if needed)

Example:

```
{
    int c = a + b;
    return c;
}
```

7. Return Statement

- Used to send a value back to the calling function.
- Required if the return type is not void.

Example:

```
return a + b;
```

8. Function Call

- The point where the function is invoked (executed) from main() or another function.
- Requires matching the function's signature (name, parameter count and type).

Example:

```
int result = add(5, 10);
```


5. What is Recursion. Explain the process of Recursion in detail.

Definition:

Recursion is a process in which a **function calls itself** to solve a problem. The function that calls itself is known as a **recursive function**.

Recursion is used to break a problem into **smaller subproblems** of the same type until a **base case** is reached.

Process of Recursion

A recursive function has **two main parts**:

1. **Base Case (Stopping Condition):**
 - Stops the recursion.
 - Prevents infinite recursion.
2. **Recursive Case:**
 - The function calls itself with **smaller or simpler input**.

Step-by-Step Example: Factorial of a Number

Factorial formula: $n! = n * (n-1)!$

```
#include <stdio.h>
```

```
int factorial(int n)
{
    if(n == 0) return 1;    // base case
    return n * factorial(n - 1); // recursive call
}
```

```
int main() {
    int num = 5;
    printf("Factorial of %d is %d", num, factorial(num));
    return 0;
}
```

Output:

Factorial of 5 is 120

Recursion Flow for factorial(3):

factorial(3) → 3 * factorial(2)

factorial(2) → 2 * factorial(1)

factorial(1) → 1 * factorial(0)

factorial(0) → 1 (base case)

Unwinding: 1 → 2 → 6

Types of Recursion

1.1 Tail Recursion

- The recursive call is the **last statement** in the function.
- Nothing happens after the recursive call returns.

Example: Tail Recursion

```
#include <stdio.h>
```

```
void printNumbers(int n) {
```

```

    if(n == 0) return;    // base case
    printf("%d ", n);
    printNumbers(n - 1); // recursive call
}

```

```

int main() {
    printNumbers(5);
    return 0;
}

```

Output:

5 4 3 2 1

1.2 Non-Tail Recursion

- The function performs **some operations after the recursive call** returns.

Example: Non-Tail Recursion

```
#include <stdio.h>
```

```

void printNumbers(int n) {
    if(n == 0) return;
    printNumbers(n - 1); // recursive call
    printf("%d ", n);    // operation after recursive call
}

```

```

int main() {
    printNumbers(5);
    return 0;
}

```

Output:

1 2 3 4 5

3. Features of Recursion

- Requires at least **one base case**.
- Does **not use looping statements** (for, while).
- Can solve problems that can be broken into smaller similar subproblems.

4. Advantages of Recursion

- **Simpler and elegant code** for complex problems.
- Useful for **mathematical and algorithmic problems**:
 - Factorial
 - Fibonacci series
 - Tower of Hanoi
 - Tree and graph traversals
- **Reduces code size** and increases readability.

5. Disadvantages of Recursion

- **Slower than loops** due to repeated function calls.
- **Memory-intensive** – uses stack memory for each call.
- **Risk of infinite recursion** if the base case is not defined.

Example: Fibonacci Series

```
#include <stdio.h>

int fibonacci(int n) {
    if(n == 0) return 0;    // base case 1
    if(n == 1) return 1;    // base case 2
    return fibonacci(n - 1) + fibonacci(n - 2); // recursive call
}

int main() {
    int n = 7;
    for(int i = 0; i < n; i++)
        printf("%d ", fibonacci(i));
    return 0;
}
```

Output:

0 1 1 2 3 5 8

6. Explain in Detail Storage Classes in C

Definition:

Storage classes in C determine the scope, visibility, lifetime, and memory location of a variable. C provides four main storage classes:

1. auto
 2. register
 3. static
 4. extern
-

1. auto Storage Class

Description

- Default storage class for local variables inside a function.
- Scope: Within the function/block in which it is defined.
- Lifetime: Exists only during function execution.
- Stored in stack memory.

Syntax

auto int x; // optional, as local variables are auto by default

Example

```
#include <stdio.h>
```

```
int main() {
    auto int x = 10; // auto is optional
    printf("%d", x);
    return 0;
}
```

Notes:

- Local variable x is destroyed after main() ends.
- If you declare without auto, it behaves the same.

2. register Storage Class

Description

- Suggests storing the variable in CPU registers for faster access.
- Scope: Within the block.
- Lifetime: During function execution.
- Cannot take address using & operator.

Syntax

register int counter;

Example

```
#include <stdio.h>
```

```
int main() {  
    register int i;  
    for(i = 1; i <= 5; i++) {  
        printf("%d ", i);  
    }  
    return 0;  
}
```

Notes:

- Used for frequently used variables like loop counters.
- Modern compilers may ignore register and optimize automatically.

3. static Storage Class

Description

- Preserves the value of a variable between function calls.
- Scope can be local or global.
- Lifetime: Entire program execution.
- Stored in data segment (not stack).

Syntax

static int count = 0;

Example 1: Static Local Variable

```
#include <stdio.h>
```

```
void demo() {  
    static int count = 0; // retains value across calls  
    count++;  
    printf("Count = %d\n", count);  
}
```

```
int main() {  
    demo();  
    demo();  
    demo();  
    return 0;  
}
```

Output:

Count = 1

Count = 2

Count = 3

Example 2: Static Global Variable

```
static int x = 10; // accessible only in this file
```

Notes:

- Static limits visibility to the file if declared globally.
- Useful for preserving values in functions.

4. extern Storage Class

Description

- Refers to a global variable defined in another file or location.
- Scope: Global.
- Lifetime: Entire program execution.
- Used to share variables across files.

Syntax

```
extern int x;
```

Example

```
#include <stdio.h>
```

```
extern int x; // defined in another file or above main()
```

```
int main() {  
    printf("%d", x);  
    return 0;  
}
```

```
int x = 50; // definition
```

Notes:

- Only declares the variable; actual definition must exist somewhere.
- Useful in multi-file programs.

5. Summary Table

Storage Class	Scope	Lifetime	Default Value	Memory Location	Example
auto	Local	Function execution	Garbage	Stack	auto int x;
register	Local	Function execution	Garbage	CPU Register	register int i;
static	Local/Global	Entire program	0	Data Segment	static int count=0;
extern	Global	Entire program	N/A	Data Segment	extern int x;

Storage Classes and Memory Allocation

Storage Class	Scope	Lifetime	Memory Location
auto	Local	Function execution	Stack
register	Local	Function execution	CPU Register (or Stack)
static	Local/Global	Entire program	Data Segment
extern	Global (across files)	Entire program	Data Segment

5. Explain in Detail the Scope and Lifetime of the Variables.

In C, variables have two important properties:

1. **Scope** – where the variable can be **accessed** in the program.
2. **Lifetime** – how long the variable **exists in memory** during program execution.

Scope of Variables

Definition:

Scope refers to the **region of the program** where a variable can be **used or accessed**.

Types of Scope

Type	Description	Example
Local Scope	Variable declared inside a function or block ; accessible only within that function/block.	<code>c void func() { int x = 5; // x accessible only here }</code>
Global Scope	Variable declared outside all functions ; accessible throughout the program after declaration.	<code>c int x = 10; void func() { printf("%d", x); }</code>
Block Scope	Variable declared inside { } block; accessible only within that block .	<code>c { int y = 20; // y accessible only here }</code>
Function Scope	Labels used with goto; accessible within the function .	<code>c func: printf("Hello"); goto func;</code>

3. Lifetime of Variables

Definition:

Lifetime refers to the **duration** for which a variable **exists in memory** during program execution.

Types of Lifetime

Type	Description	Storage Class Example
Automatic (temporary)	Exists only during function execution ; memory released after function ends.	<code>auto int x; (default local variable)</code>
Static (permanent)	Exists throughout the program execution ; retains value between function calls.	<code>static int count;</code>
Dynamic	Allocated and deallocated manually during runtime using malloc/free.	<code>int *ptr = malloc(sizeof(int));</code>

Type	Description	Storage Class Example
Global	Exists throughout the program execution ; accessible in all functions (or with extern).	int x; declared outside functions

3. Relationship Between Scope and Lifetime

Scope	Lifetime	Notes
Local (auto)	Temporary	Accessible only in function; destroyed after execution
Static (local)	Permanent	Accessible only in function; retains value across calls
Global	Permanent	Accessible in entire program; exists until program ends
Dynamic	Manual	Can be accessed via pointer; exists until freed

4. Examples

Local Variable

```
#include <stdio.h>
void demo() {
    int x = 10; // local scope, auto lifetime
    printf("%d\n", x);
}
int main() {
    demo();
    // x not accessible here
    return 0; }
```

Static Variable

```
#include <stdio.h>
void demo() {
    static int count = 0; // retains value across calls
    count++;
    printf("Count = %d\n", count);
}
int main() {
    demo(); // Count = 1
    demo(); // Count = 2
    demo(); // Count = 3
    return 0;
}
```

Global Variable

```
#include <stdio.h>
int x = 50; // global variable
void demo() {
    printf("%d\n", x); // accessible here
}
int main() {
    demo();
    printf("%d\n", x); // accessible here too
    return 0;
}
```

5. Summary Table

Variable Type	Scope	Lifetime	Example Storage Class
Local	Function/block	Temporary	auto (default)
Static Local	Function/block	Entire program	static
Global	Entire program	Entire program	global, extern
Dynamic	Accessible via pointer	Manual (malloc/free)	n/a

Summary:

1. **Scope determines accessibility**, lifetime determines **existence in memory**.
2. **Static variables** have limited scope but **extended lifetime**.
3. **Local auto variables** have limited scope and lifetime.
4. **Global variables** have maximum scope and lifetime.